

บทที่ 1

แนวคิดและพื้นฐานการเขียนโปรแกรมภาษา Python

Concepts and Fundamentals of Programming in Python

อาจารย์ผู้สอน:

- ▶ อ. ดร.พงษ์ศร จันทรียอย
วิทยาลัยการคอมพิวเตอร์ มหาวิทยาลัยขอนแก่น



PYTHON

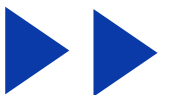


วัตถุประสงค์การเรียนรู้

- เพื่อให้ผู้เรียนเข้าใจหลักการเขียนโปรแกรมคอมพิวเตอร์ขั้นพื้นฐาน
- เพื่อให้ผู้เรียนสามารถเขียน และรันโปรแกรมภาษา Python ได้
- เพื่อให้ผู้เรียนสามารถใช้คำสั่งนำเข้าและแสดงผลได้

เนื้อหา

01. ความรู้พื้นฐานเกี่ยวกับการเขียนโปรแกรม
02. แนะนำภาษา Python
03. การติดตั้งใช้งาน Python
04. คำสั่งแสดงผลออกทางจอภาพ `print()`
05. คำสั่งรับค่าจากแป้นพิมพ์ `input()`



โปรแกรมคืออะไร?

- โปรแกรม คือ ชุดของคำสั่งที่เขียนขึ้นมาด้วยภาษาโปรแกรม เพื่อให้คอมพิวเตอร์ประมวลผลตามวัตถุประสงค์ต้องการ
- องค์ประกอบของโปรแกรม



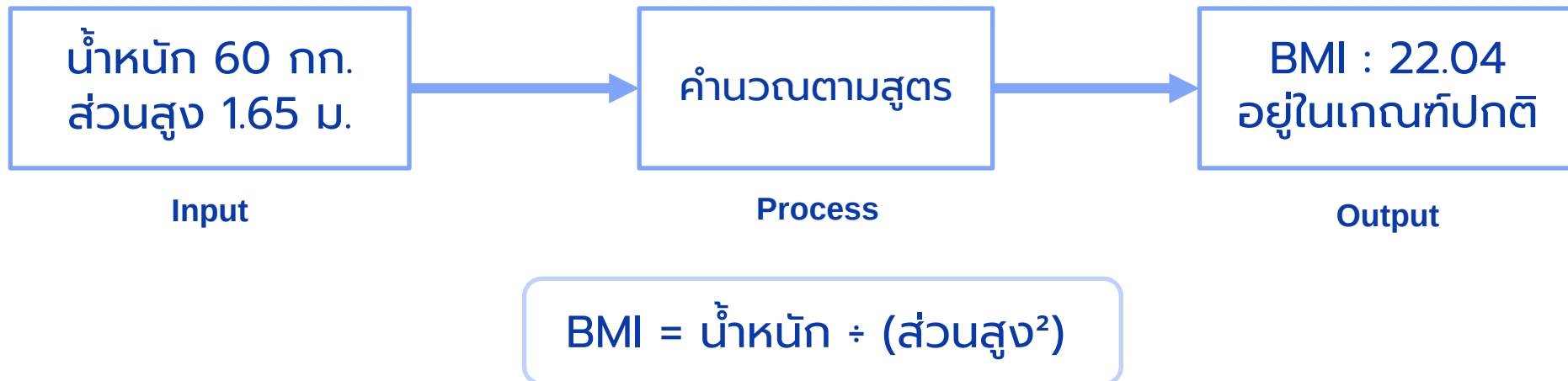
- รับข้อมูล (Input)
 - รับข้อมูลจากผู้ใช้งาน เช่น การป้อนผ่านแป้นพิมพ์ การคลิกเมาส์หรือกดปุ่มบนหน้าจอ
- ประมวลผล (Process)
 - นำข้อมูลมาคำนวณหรือตัดสินใจตามเงื่อนไขที่กำหนดไว้
- ผลลัพธ์ (Output)
 - แสดงผลลัพธ์ที่ได้จากการประมวลผล เช่น แสดงทางจอภาพ พิมพ์ออกทางเครื่องพิมพ์

โปรแกรมคืออะไร?

- ตัวอย่างโปรแกรม

- โปรแกรมคำนวณดัชนีมวลกาย (BMI):

- น้ำหนัก (กิโลกรัม) ส่วนสูง (เมตร) → คำนวณตามสูตร → แสดง BMI



โปรแกรมคืออะไร?

- ตัวอย่างโปรแกรม :
 - โปรแกรมคำนวณเกรด:
 - รับคะแนน → คำนวณเกรดตามเกณฑ์ → แสดงเกรด



ช่วงคะแนน	เกรด
คะแนน ≥ 80	A
$70 \leq \text{คะแนน} < 80$	B
$60 \leq \text{คะแนน} < 70$	C
$50 \leq \text{คะแนน} < 60$	D
คะแนน < 50	F



Question #1:



Question #2:



ทำไมต้องเขียน
โปรแกรม?



ทำไมต้องเขียนโปรแกรม?

- แก้ปัญหาและตอบโจทย์ความต้องการ
- ประหยัดเวลาและแรงงาน
 - ในการทำงานที่ต้องทำซ้ำๆ หรือใช้เวลานาน
- มีความแม่นยำสูง
 - คอมพิวเตอร์ไม่เหนื่อย ไม่เบื่อ ไม่ประมาณ สามารถทำงานซ้ำๆ ได้โดยไม่ผิดพลาด
- ทำงานที่มนุษย์ทำไม่ได้หรือทำได้ยาก
 - ประมวลผลข้อมูลขนาดใหญ่ (Big Data) การวิเคราะห์รูปภาพหรือเสียง จำลองสถานการณ์ที่ซับซ้อน
- สร้างสรรค์นวัตกรรมใหม่ๆ
 - แอปพลิเคชันมือถือ (Mobile Apps) เกม (Games) และปัญญาประดิษฐ์ (Artificial Intelligence)



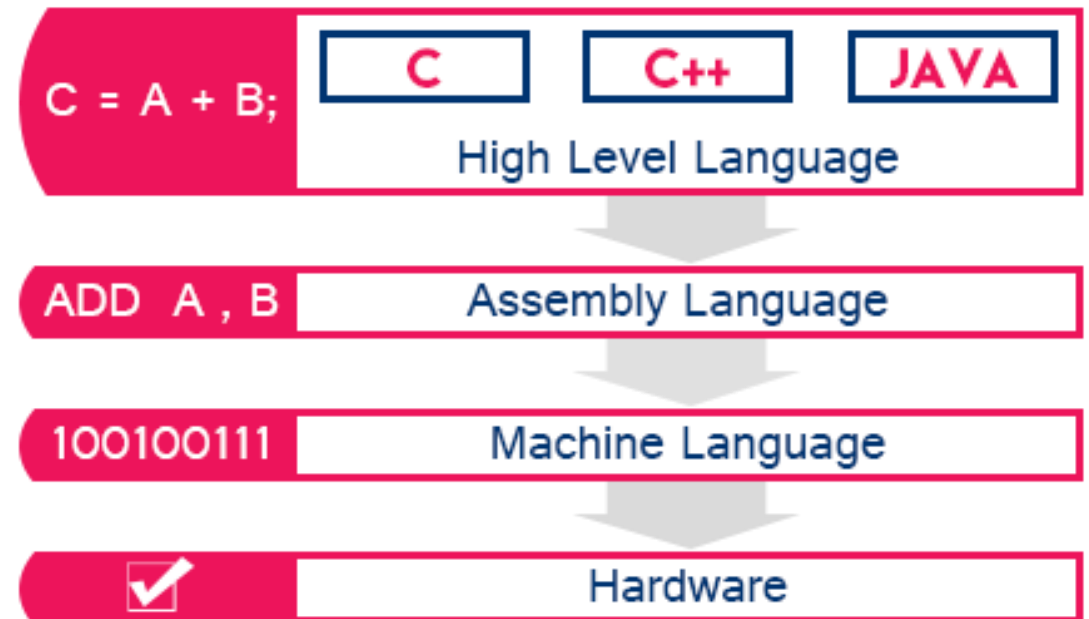


เคยสงสัยไหมว่า
โปรแกรมที่เราเขียน
ทำงานได้อย่างไร?



Compiler vs Interpreter

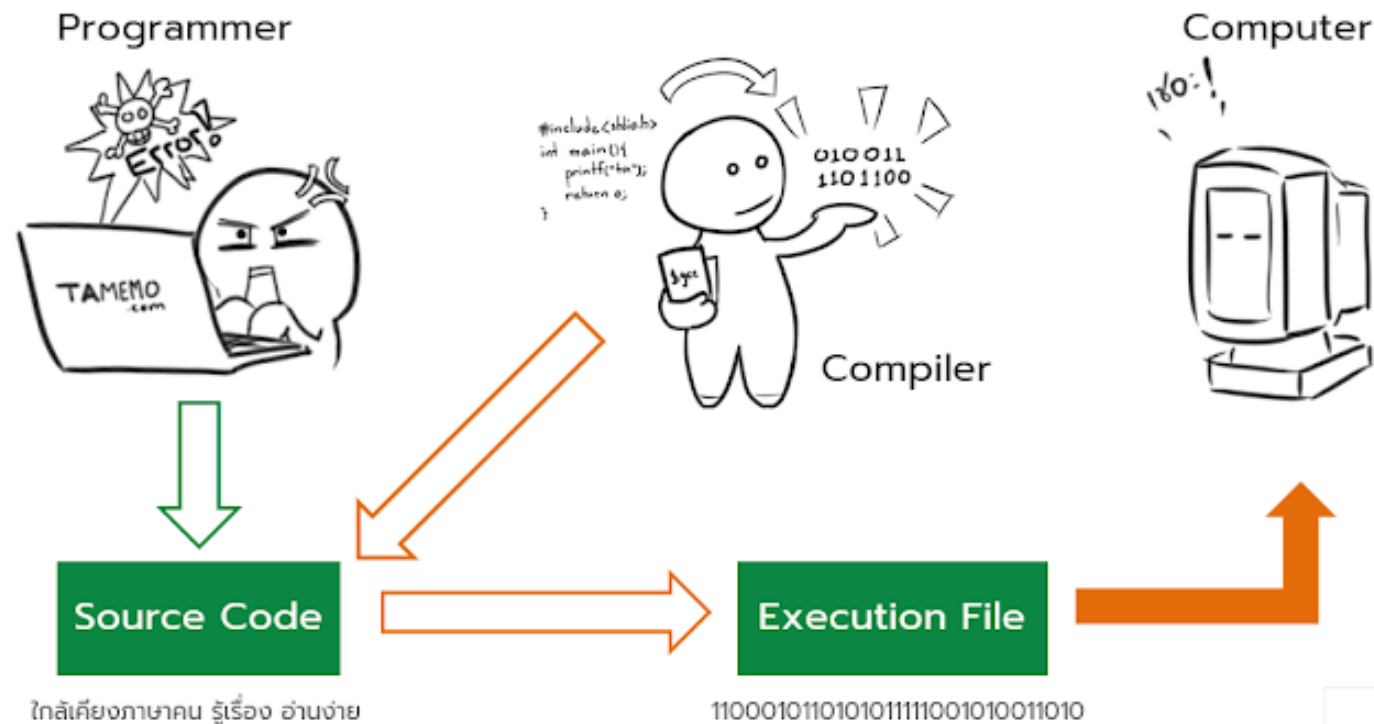
- **ภาษาระดับสูง (High-level language)**
 - ภาษาที่ใช้เขียนโปรแกรม
 - Python, C, Java, JavaScript เป็นต้น
 - มนุษย์ (programmer) อ่านแล้วเข้าใจ
- **ภาษาระดับต่ำ (Low-level language)**
 - Assembly languages
 - ยังใกล้เคียงกับภาษามนุษย์
- **ภาษาเครื่อง (Machine language)**
 - คอมพิวเตอร์อ่านแล้วเข้าใจ
 - มีลักษณะเป็นเลขฐานสอง (binary numbers)
 - ประมวลผลตามคำสั่งที่เขียนไว้



Compiler vs Interpreter

- **Compiler และ Interpreter**

- ทำหน้าที่แปลงโค้ดโปรแกรมที่เขียนไว้ด้วยภาษาระดับสูง ให้คอมพิวเตอร์เข้าใจ



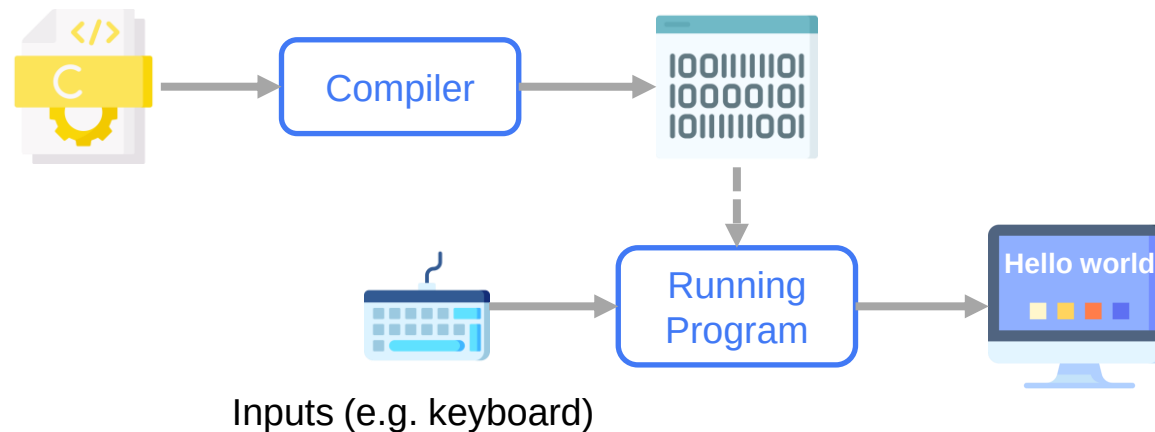
Compiler vs Interpreter

• คอมไพเลอร์ (Compiler)

- ทำหน้าที่ Compile หรือ แปลงโค้ดของโปรแกรมทั้งหมด ให้เป็น Execution File
- ถ้าจะให้โปรแกรมทำงานก็สั่งรัน Execution File ที่ได้อีกครั้ง
- ตัวอย่างภาษาในกลุ่มนี้ เช่น C, C++, C#

เราจะเรียกสิ่งนี้ว่า Execution file หรือที่เรียกติดปากว่าไฟล์ .exe

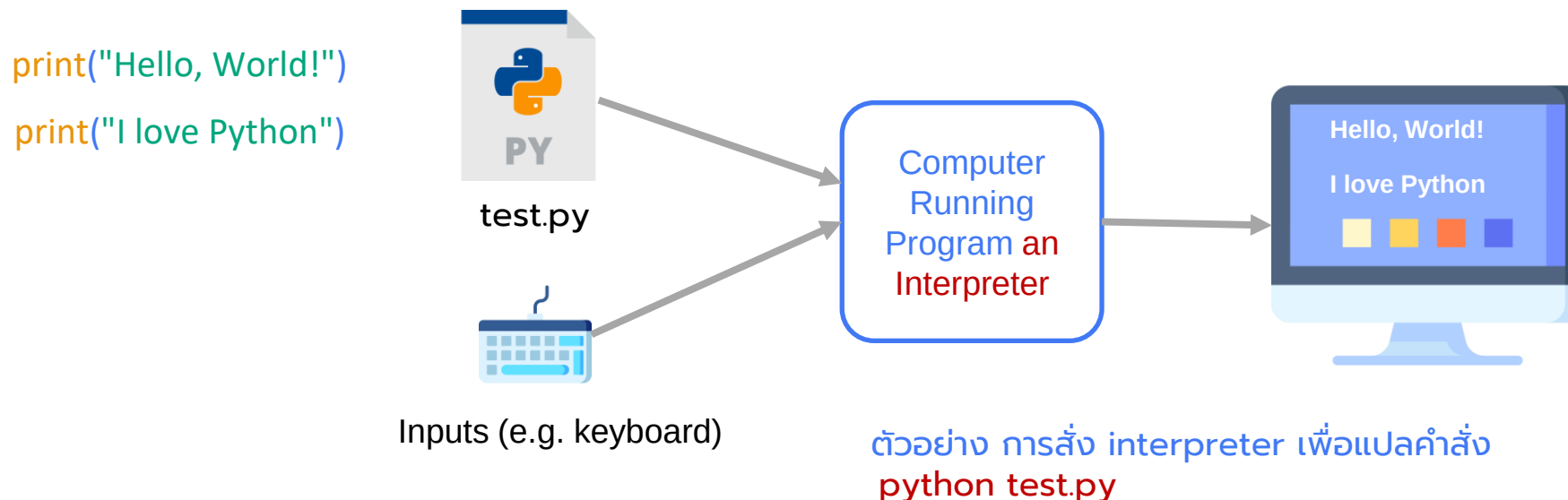
```
#include <stdio.h>
int main() {
    printf("Hello, World!");
    return 0;
}
```



Compiler vs Interpreter

• อินเทอร์พรีเตอร์ (Interpreter)

- แปลงโค้ดเหมือน Compiler แต่จะไม่แปลงโค้ดทีเดียวทั้งโปรแกรม
- แต่จะแปลคำสั่งทีละบรรทัด ทำไปเรื่อยๆ จนจบโปรแกรม
- ตัวอย่างภาษาในกลุ่มนี้ เช่น Python, PHP, Ruby, Bash





Question #3



Question #4



แนะนำภาษา Python

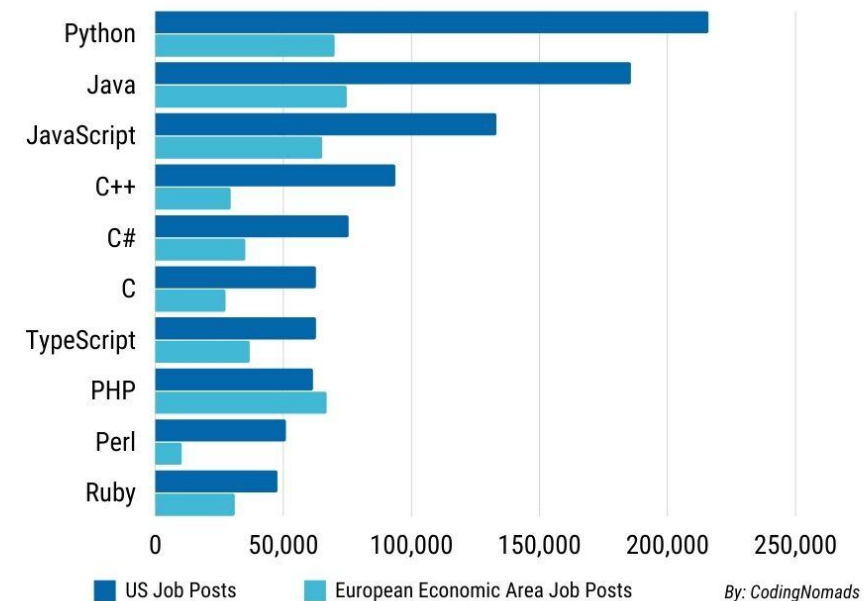


Python Programming Language

- **Python**
 - เป็นภาษาระดับสูงที่ใช้ในการเขียนโปรแกรม
 - มีลักษณะเป็นภาษาสคริปต์ ซึ่งจะประมวลผลทีละบรรทัด (Interpreter-based language)
 - สร้างโดย Guido van Rossum โปรแกรมเมอร์ชาวเนเธอร์แลนด์ ในปี 1989

Most in-demand programming languages of 2022

Based on LinkedIn job postings in the USA & Europe



<https://codingnomads.co/blog/the-best-programming-languages-to-learn/>

Python Programming Language

- **ลักษณะเด่นของ Python**

- เรียนรู้ได้ง่าย ภาษา Python มีโครงสร้างที่ไม่ซับซ้อน
- มีไลบรารี (Library) ที่หลากหลาย
 - Machine Learning
 - Data Analytics
 - Web Application
 - System Admin
- สามารถเขียนโปรแกรมได้หลายกระบวนทัศน์ (Paradigm)
 - Structured programming
 - Object-Oriented Programming
 - Functional Programming

Python Programming Language

- **ลักษณะเด่นของ Python**

- ทำงานได้หลายระบบปฏิบัติการ (Operating system: OS)
- มีแหล่งเรียนรู้จำนวนมาก มีชุมชนผู้ใช้งานขนาดใหญ่ มีเอกสาร บทความ และคอร์สเรียนมากมาย

Prominent examples of companies using Python



Python Programming Language

- Python ทำอะไรได้บ้าง?



<https://www.abtosoftware.com/blog/working-with-pythons-fastapi>

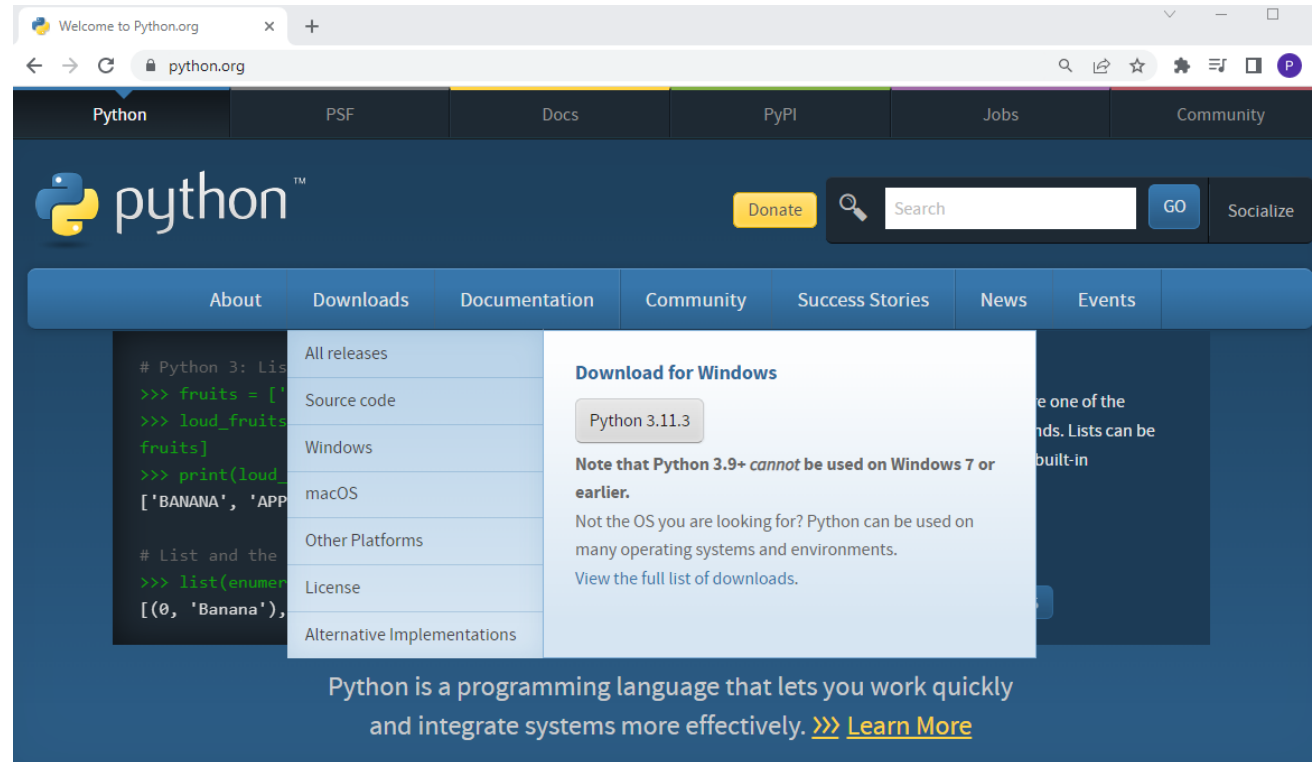


การติดตั้งและใช้งาน ภาษา Python



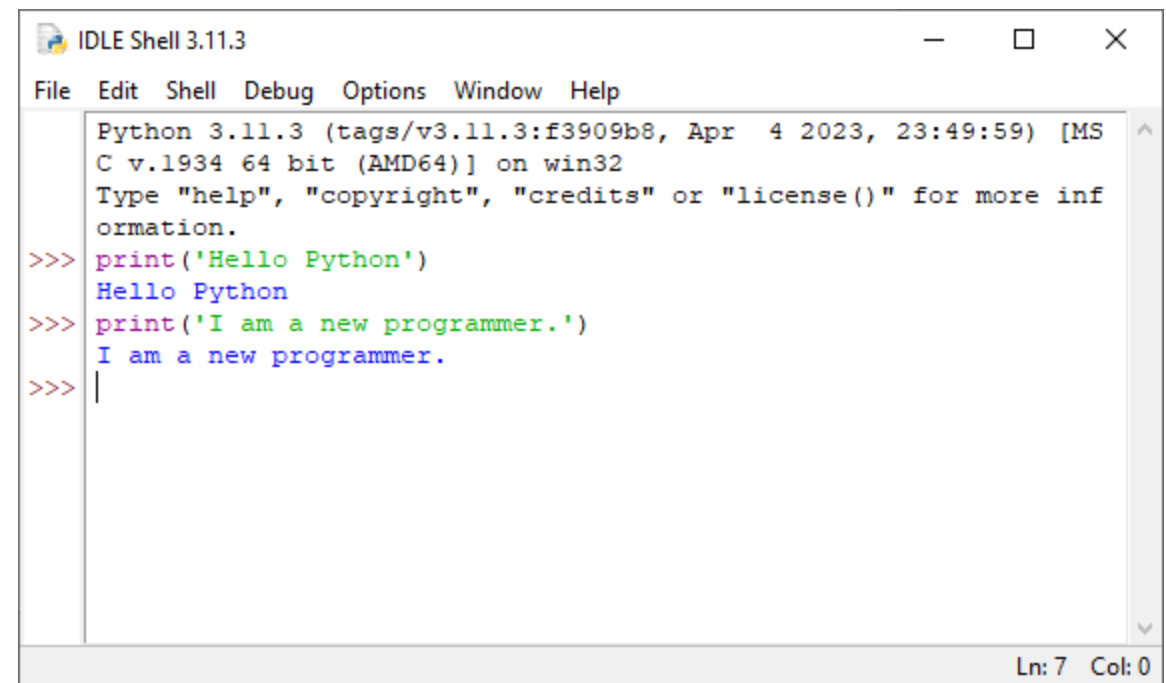
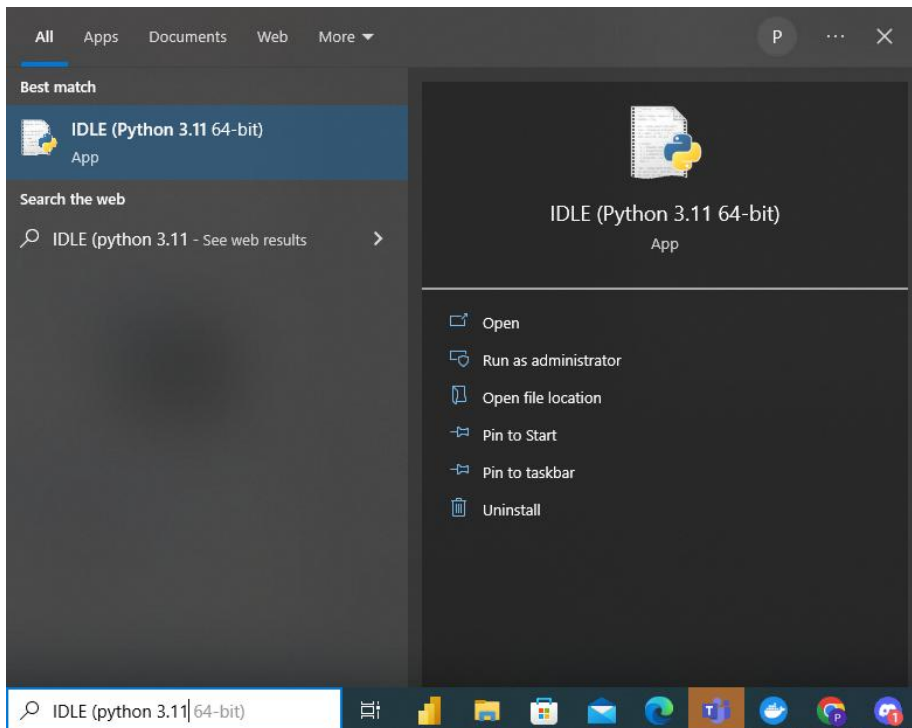
การติดตั้ง Python

- Download ไฟล์ติดตั้งจาก <https://www.python.org/>
- เปิดไฟล์เพื่อติดตั้ง Python



ทดสอบตัวแปลภาษา Python

- พืพม์ **IDLE (Python 3.11 64-bit)** ที่ช่องค้นหา และเปิดโปรแกรม
- ลองพิมพ์คำสั่งเพื่อทดสอบ



การรับโค้ดภาษา Python ผ่าน Command line

- ในการเขียนโปรแกรม โค้ดของโปรแกรมจะถูกเก็บเป็นไฟล์ ซึ่งอาจจะมีหลายไฟล์ก็ได้
- ลองสร้างไฟล์ test.py แล้วเพิ่มโค้ดดังนี้

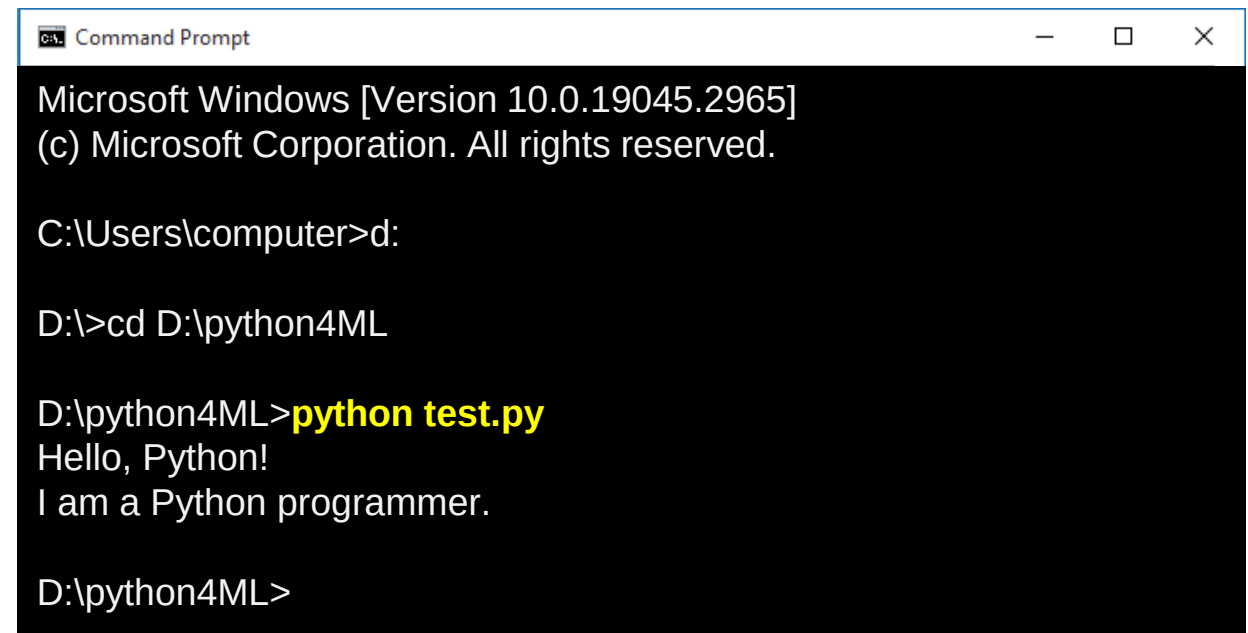
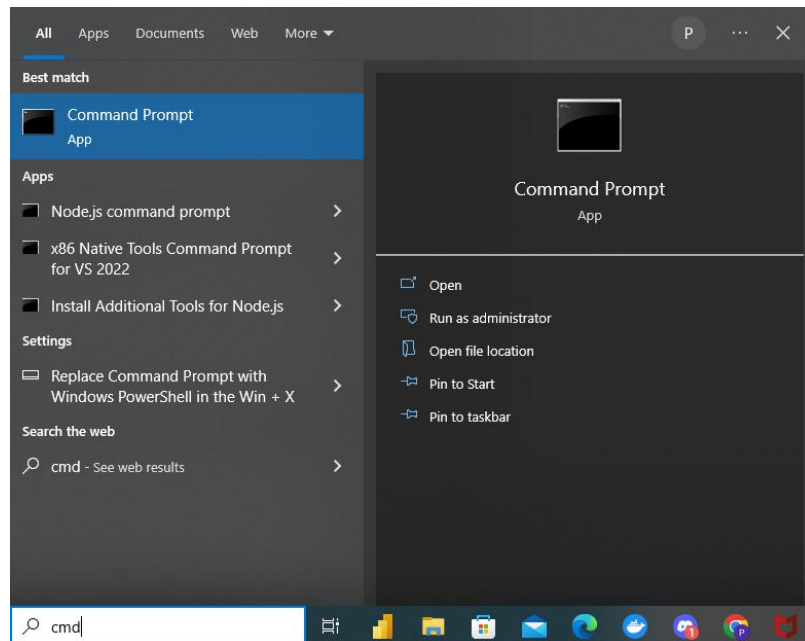


test.py

```
print('Hello, Python!')  
print("I am a Python programmer.")
```

การรันโค้ดภาษา Python ผ่าน Command line

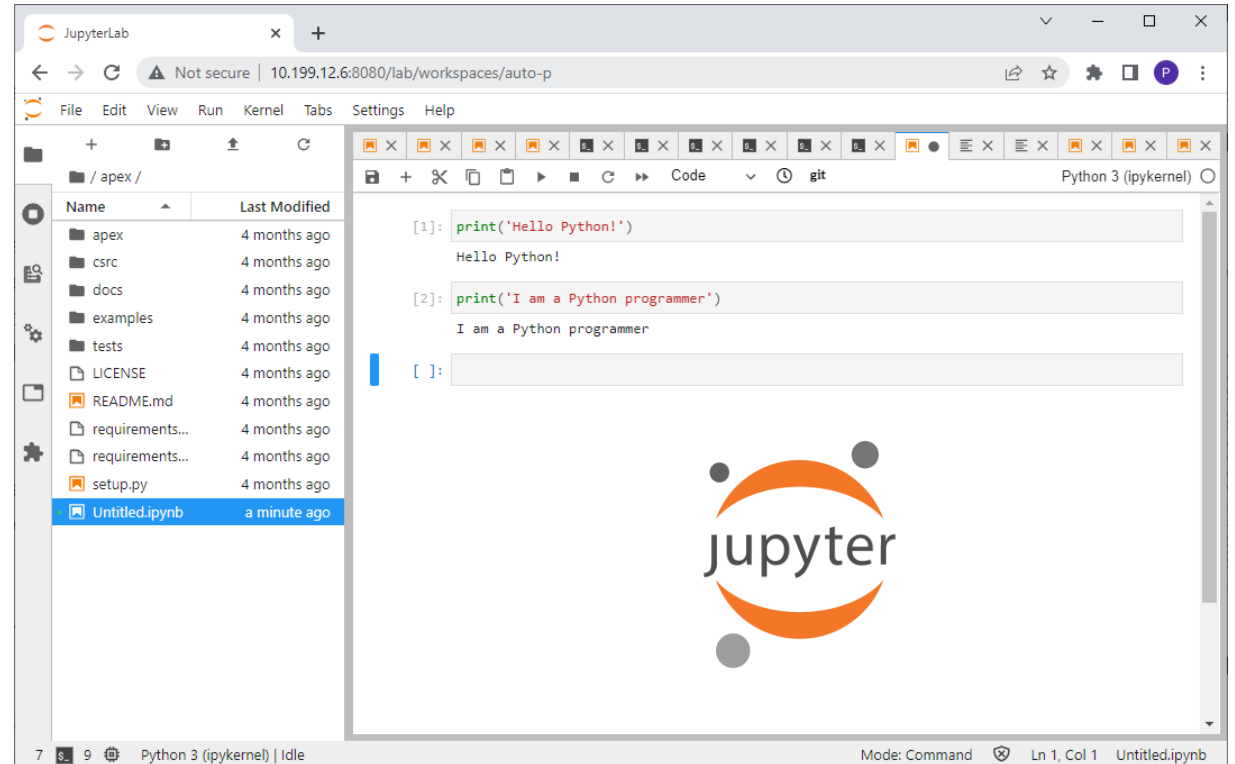
- เปิดโปรแกรม Command Prompt แล้วพิมพ์คำสั่ง **cmd**
- **รันโค้ด**โดยใช้คำสั่ง:
 - `python test.py`



cd – เป็นคำสั่งสำหรับเป็นไดเรกทอรีสำหรับทำงาน (Change the working directory)

การรันโค้ดภาษา Python บน Browser

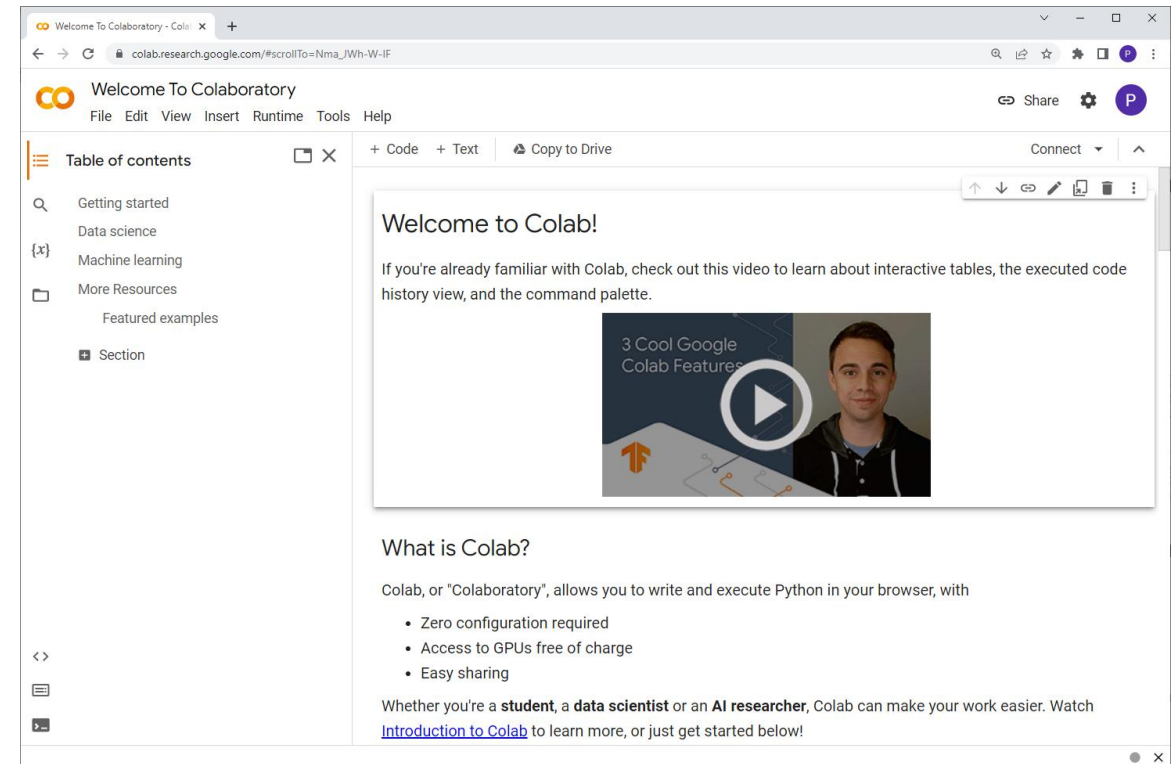
- **Jupyter Lab: <https://jupyter.org/>**
 - เขียนโค้ดและรันผ่าน Web browser (Chrome, Firefox, Edge)
 - สามารถรันโค้ดทีละส่วน Interactive features, code re-execution, result storage
 - สะดวกในการทดสอบโปรแกรม
 - ติดตั้งและใช้งานโดยใช้คำสั่ง:
 - `pip install jupyterlab`
 - `jupyter-lab`



<https://jupyter.org/install>

การรันโค้ดภาษา Python บน Browser

- **Colab:**
<https://colab.research.google.com/>
 - เขียนโค้ดและรันผ่าน **Web browser** (Chrome, Firefox, Edge)
 - สนับสนุนการทำงานร่วมกัน (Real-time collaboration)
 - สนับสนุนการใช้งานไฟล์ใน **Google Drive**
 - ใช้งานทรัพยากรพื้นฐานได้**ฟรี**
 - ใช้งาน GPU และ TPUs ได้ (แต่ไม่ฟรี)
 - **ไม่ต้องติดตั้ง**เข้าใช้งานได้เลย



<https://colab.research.google.com/>



คำสั่งการแสดงผล Print Command



คำสั่งแสดงผล

- **print()** ใช้แสดงผลออกทางหน้าจอ
 - เป็นฟังก์ชันแบบ build-in (ติดมากับตอน Python ไม่ต้องติดตั้งเพิ่ม)
 - ฟังก์ชัน print() จะนำ object ไปแสดงผล
 - ในภาษา Python ข้อความจะครอบด้วย ' ' (single quotes) หรือ " " (double quotes)



Python Code : Example 1

```
print('Hello, Python!')
print("I am a Python programmer.")
```



ข้อความ/สายอักขระ (String)



Output

```
Hello, Python!
I am a Python programmer.
```

คำสั่งแสดงผล

- การแสดงข้อความหลายๆ บรรทัด

- สามารถใช้ `"""` (triple double quotes) หรือ `'''` (triple single quotes) กรอบข้อความที่มีหลายๆ บรรทัด

Python Code : Example 1

```
print ('Routine:')
print ('- Eat')
print ('- Study')
print ('- Sleep')
print ('- Repeat')
```

Python Code : Example 2

```
print("""Routine:
- Eat
- Study
- Sleep
- Repeat
""")
```

Python Code : Example 3

```
print('''Routine:
- Eat
- Study
- Sleep
- Repeat
''')
```

Output

```
Routine:
- Eat
- Study
- Sleep
- Repeat
```

คำสั่งแสดงผล

- **อักขระพิเศษ (Escape Characters)**
 - ตัวอักษรพิเศษ ที่ไม่สามารถกำหนดหรือแสดงอยู่ใน String เหมือนกับที่เป็นได้
 - Escape character จะเริ่มต้นด้วย **\ (backslash)** แล้วตามด้วยตัวอักษรที่ต้องการแสดง



Python Code : Example 1

```
print("I am a "Python" programmer.")
print('I 'm a "Python" programmer.')
```

Escape character	Description
\b	Backspace
\t	Tab
\n	Newline
\v	Vertical tab
\f	Form feed
\r	Carriage return
\"	Double quote
\'	Single quote
\\	Backslash

คำสั่งแสดงผล

- ตัวอย่างการใช้ **Escape Characters**
 - แบบไม่ใช้ Escape Characters
 - ใช้ Escape Characters \n

Python Code : Example 1

```
print("Monday")
print("Tuesday")
print("Wednesday")
print("Thursday")
print("Friday")
print("Saturday")
print("Sunday")
```

Python Code : Example 2

```
print("Monday\nTuesday\nWednesday\nThursday\nFriday\nSaturday\nSunday")
```

Output

```
Monday
Tuesday
Wednesday
Thursday
Friday
Saturday
```



Question #5



Question #6

คำสั่งแสดงผล

- พารามิเตอร์ **end** และ **sep** ในคำสั่ง **print()**

```
print(object(s), sep=separator, end=end, file=file, flush=flush)
```

- end** – ใช้กำหนดตัวอักษรหรือข้อความต่อท้ายหลังจากแสดงข้อความ ถ้าไม่กำหนดจะถูกกำหนดเป็นค่า default คือ \n หมายถึงให้ขึ้นบรรทัดใหม่
- sep** - ใช้กำหนดตัวคั่นกรณีแสดงหลายข้อความพร้อมกัน โดยค่า default จะเป็น ช่องว่าง

Python Code : Example 1

```
print("Welcome to", end = ' ')
print("Python", end = '!')
print(" Welcome", "to", "Python")
print("Welcome", "to", "Python", sep=' - ')
```

Output

```
Welcome to Python! Welcome to Python
Welcome - to - Python
```

คำสั่งแสดงผล

- ใช้ฟังก์ชัน `print()` แสดงค่าของตัวแปร

Python Code : Example 1

```
first_name="Pongsathon"  
last_name="Janyoi"  
age=34  
gpa=3.81  
  
print('My name is ',first_name,last_name)  
print('I am ',age,'years old')  
print('GPA :',gpa)
```

Output

```
My name is  Pongsathon Janyoi  
I am  34 years old  
GPA : 3.81
```



คำสั่งรับค่าจาก
แป้นพิมพ์
Input()



คำสั่งรับค่าจากแป้นพิมพ์

- **input()**
 - เป็นคำสั่งสำหรับรับค่าจากผู้ใช้งานจากทางแป้นพิมพ์
 - ไวยากรณ์ (Syntax)

```
var_name = input([prompt])
```

- **prompt** - ข้อความที่แสดงบนหน้าจอเพื่ออธิบายผู้ใช่ว่าให้ป้อนข้อมูลอะไร (ไม่กำหนดก็ได้)
- **var_name** - ตัวแปรที่ใช้เก็บข้อมูลที่รับมา ตัวแปรนี้จะป็นชนิดข้อความ (String)

คำสั่งรับค่าจากแป้นพิมพ์

- **input()**
 - ตัวแปรที่ใช้เก็บข้อมูลจะมีชนิดเป็นข้อความ
 - สามารถตรวจสอบชนิดตัวแปรด้วยฟังก์ชัน **type()**

Python Code : Example 1

```
name=input('Enter your name: ')\nage=input('How old are you?:')\n\nprint()\nprint('My name is ',name)\nprint(type(name))\n\nprint('I am ',age,'years old. ')\nprint(type(age))
```

Output

```
Enter your name: Manman\nHow old are you?:34\n\nMy name is  Manman\n<class 'str'>\nI am  34 years old.\n<class 'str'>
```

คำสั่งรับค่าจากแป้นพิมพ์

- **input()**
 - หากต้องการนำค่าไปคำนวณทางคณิตศาสตร์ จะต้องแปลงชนิดข้อมูล (Casting) เป็นเลขจำนวนจริงหรือจำนวนเต็มก่อน

Python Code : Example 1

```
number1 = input("Enter first number: ")
number2 = input("Enter second number: ")

print()
print("First Number: ", number1)
print("Second Number: ", number2)
sum1 = number1 + number2
print("Addition of two number is: ", sum1)
```

Output

```
Enter first number: 10
Enter second number: 20
```

```
First Number: 10
Second Number: 20
Addition of two number is: 1020
```

อ้าว!! มันเอาไปต่อกันเฉยเลย

คำสั่งรับค่าจากแป้นพิมพ์

- **input()**
 - แปลงข้อความเป็นค่าตัวเลข โดยใช้ฟังก์ชัน build-in
 - **int()**: แปลงข้อความเป็นตัวเลขจำนวนเต็ม
 - **float()**: แปลงข้อความเป็นตัวเลขทศนิยม

Python Code : Example 1

```
number1 = input("Enter first number: ")
number2 = input("Enter second number: ")

print()
print("First Number: ", number1)
print("Second Number: ", number2)
sum1 = float(number1) + int(number2)
print(type(sum1))
print("Addition of two number is: ", sum1)
```

Output

```
Enter first number: 20
Enter second number: 10

First Number: 20
Second Number: 10
<class 'float'>
Addition of two number is: 30.0
```

เย้!! ถูกแล้ว

ตัวอย่าง: โมเดลเชิงเส้น (Linear Model)

- ตัวอย่าง: การทำนายราคาบ้านด้วยการถดถอยเชิงเส้น

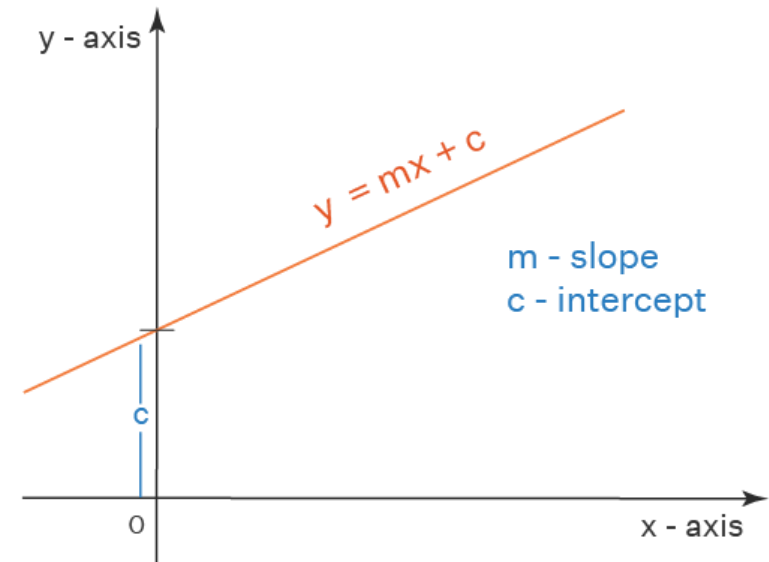
Python Code

```
x = input("Enter house area (square meters): ")
x = float(x)

y = 0.035 * x + 0.5
print("--- Prediction Result ---")
print("House area:", x, "square meters")
print("Predicted price:", y, "million baht")
```

Output

```
Enter house area (square meters): 100
--- Prediction Result ---
House area: 100.0 square meters
Predicted price: 4.0 million baht
```



$$\text{ราคา} = 0.035 \times \text{พื้นที่} + 0.5$$

การละเว้นคำสั่ง (Comment)

- การคอมเมนต์เป็นสิ่งที่สำคัญอย่างหนึ่งในการเขียนโปรแกรม
 - ใช้อธิบายโค้ดส่วนต่างๆ ที่เขียน ช่วยลดเวลาในการแก้โค้ด และยังเป็นประโยชน์สำหรับผู้พัฒนาต่อ
 - ใน Python เราจะใช้เครื่องหมาย **#** เพื่อละเว้นโค้ดในบรรทัดนั้นไม่ให้ประมวลผล

 Python Code : Example 1

```
n=10  
#n+=1  
n+=2 #add n by 2  
print(n)
```

 Output

12

การละเว้นคำสั่ง (Comment)

- การคอมเมนต์หลายบรรทัด

- สามารถใช้ `'''` (Triple Single Quotes) หรือ `"""` (Triple Double Quotes) ครอบโค้ดส่วนที่ต้องการคอมเมนต์หลายบรรทัด

 Python Code : Example 1

```
#This is also a  
#perfect example of  
#multi-line comments
```

```
score=50  
print(score)
```

 Python Code : Example 1

```
"""  
This is also a  
perfect example of  
multi-line comments  
"""  
score=50  
print(score)
```

 Output

```
50
```

 Question #7



โหมดการช่วยเหลือ (Help mode)



โหมดการช่วยเหลือ

- หากไม่เข้าใจเกี่ยวกับคำสั่งต่างๆ ก็สามารถใช้โหมดการช่วยเหลือได้
- สามารถเรียกใช้โดยใช้คำสั่ง `help('ชื่อฟังก์ชัน')`

 Python Code : Example 1

```
help('print')
```

 Output

Help on built-in function print in module builtins:

print(...)

print(value, ..., sep=' ', end='\n', file=sys.stdout, flush=False)

Prints the values to a stream, or to sys.stdout by default.

Optional keyword arguments:

file: a file-like object (stream); defaults to the current sys.stdout.

sep: string inserted between values, default a space.

end: string appended after the last value, default a newline.

flush: whether to forcibly flush the stream.



Question #8

สรุปท้ายบท

- **แนะนำภาษา Python**
 - ความแตกต่างระหว่าง Compiler (ภาษา C, C++) และ Interpreter (ภาษา Python)
- **การติดตั้งและใช้งาน Python**
 - การรันไฟล์ .py, JupyterLab, Colab
- **คำสั่งแสดงผลทางหน้าจอ print()**
 - `print('I like Python.')`
- **คำสั่งรับค่าจากแป้นพิมพ์ input()**
 - `var = input("Enter something: ")`
- **การละเว้นโค้ดหรือคอมเมนต์ (Comment)**
 - ใส่ # ข้างหน้าบรรทัดที่ต้องการคอมเมนต์
- **โหมดการช่วยเหลือ (Help mode)**
 - `help('print')`